

Автономная генерация планет в Unity с Procedural Planet Generation Lite

Executive summary

Lite-версия (бесплатный «Lite Samples Pack») Procedural Planet Generation от Parallel Cascades — это набор упрощённых шейдеров и эффектов для процедурных небесных тел в Unity: «луны» (каменистые сферы с процедурной окраской), «earth-like» планеты (континенты, океаны, полярные шапки), газовые гиганты (анимированные полосы), кольца и звёзды (расплавленная поверхность + свечение). В поставке заявлены инструменты рандомизации как в Editor, так и в рантайме. ¹

Ключевое ограничение Lite: это шейдерная визуализация “нелэндабельных” планет для обзора из космоса, а не геометрическая генерация рельефа/terrain. ² В сравнении с платной версией Lite не даёт ряда важных “реализм-фич” (нормали от heightmap, динамические облака, огни городов, штормы, VFX-корона, инструменты texture baking, кастомное омни-направленное освещение и дополнительные шейдер-варианты). ¹

Для “автономной генерации планет Claude-ом” с минимумом ручных действий при обновлениях кода оптимальна стратегия: **детерминированные Planet Spec файлы → Unity batchmode генерация/обновление материалов и префабов → сборка Addressables/AssetBundles → визуальная и производительная валидация → публикация артефактов**. Такой подход снижает зависимость от ручных кликов в инспекторе и частично защищает от изменений API ассета.

Scope, источники и допущения

Основные официальные источники — документация Procedural Planet Generation и страница “Lite Samples Pack” (как “контракт возможностей/ограничений Lite”), плюс карточка ассета в Unity Asset Store ³ (версии Unity/совместимость/лицензия). ⁴

Допущения (потому что ограничения не заданы): - Unity/CI: **нет специфических ограничений** — предполагается возможность запуска Unity в batchmode на CI-раннере и хранение сгенерированных ассетов в репозитории или в билд-артефактах. - Render pipeline: ориентируемся на URP, потому что Lite отмечен как совместимый с URP и не совместимый с Built-in/HRDP на карточке ассета. ⁵ - “Автономность” Claude: подразумевается, что Claude имеет доступ к репозиторию/скриптам, но не выполняет ручные операции “в UI”; все операции должны быть скриптуемыми.

Ссылки на ключевые страницы (как просили — прямые URL в код-блоке):

```
https://parallelcascades.com/procedural-planet-generation/latest/  
https://parallelcascades.com/procedural-planet-generation/latest/lite-  
samples-pack/
```

```
https://assetstore.unity.com/packages/vfx/shaders/procedural-planet-
generation-lite-baked-planets-and-simple-proce-296362
https://parallelcascades.com/procedural-planet-generation/latest/get-started/
https://parallelcascades.com/procedural-planet-generation/latest/post-
processing-glow/
https://parallelcascades.com/procedural-planet-generation/latest/system-
requirements-and-performance-optimisation/
https://parallelcascades.com/procedural-planet-generation/latest/
antialiasing/
https://parallelcascades.com/procedural-planet-generation/latest/filtering/
https://parallelcascades.com/procedural-planet-generation/latest/known-
issues/
https://parallelcascades.com/procedural-planet-generation/latest/changelog/
```

Возможности Lite, ограничения, Unity версии, зависимости, лицензия и точки API

Что умеет Lite

Официально заявлено, что Lite включает: procedural moons, earth-like планеты (суша/океаны/полярный лёд/атмосферный glow), gas giants (полосы + атмосферные эффекты), кольца, звёзды (расплавленная поверхность + glow) и инструменты рандомизации в Editor и в рантайме. ¹

На уровне “какие параметры вообще контролируют внешний вид” (Lite): - Moons: procedurally pattern/scale + color gradient + saturation. ¹

- Earth-like (Lite): Continent scale, land-ocean coverage ratio, Ice cap extent + ice edge falloff, Noise pattern scale, land gradient, saturation, ocean deep/shallow coloration, Fresnel edge tint, atmospheric glow color/size/falloff. ¹

- Gas giant (Lite): color gradient, band scale, flow speed, базовый Fresnel edge glow. ¹

- Rings: inner/outer radius, edge fade, banding pattern, gradient/saturation, rotation speed, lighting overrides (two-sided lighting, min brightness, light falloff sharpness). ¹

- Stars: domain warp scale/intensity, pattern scale, flow speed; primary gradient + Q/R modulation; corona glow (Fresnel + unlit spherical glow density/falloff). ¹

Отдельно подчёркнуто, что сделан “Gas Giant (Simplified)” как более быстрый вариант (в документации указано “двукратная производительность” относительно дефолтного gas giant Lite) и дан путь menu item. ¹

Ограничения Lite и граница применимости

- Фреймворк ориентирован на “планеты для вида из космоса”, не на геометрию ландшабельных миров; это не terrain generation. ²
- По сравнению с платной версией Lite не включает: heightmap-based normals (реализм микрорельефа), динамические облака, city lights, storm systems, VFX-корону, инструменты static texture baking, custom omnidirectional lighting и дополнительные шейдер-варианты. ¹

Практический вывод: в Lite невозможно “честно” смоделировать, например, венерианскую непрозрачную облачную оболочку или штормы/Большое Красное Пятно Юпитера (штормы заявлены как апгрейд), но можно получить узнаваемые “палитры и силуэты” (особенно Earth/Mars/Saturn-с-кольцами). ¹

Unity версии, зависимости и совместимость

По карточке ассета в Unity Asset Store ³ : - совместимость: URP — compatible, Built-in/HDRP — not compatible; - original Unity version: 6000.0.64 (Unity 6); - license: Standard Unity Asset Store EULA; - license type: Extension Asset. ⁵

Документация Procedural Planet Generation дополнительно говорит: - URP — обязательная база; - Unity 6 требуется для Render Graph API, используемого в post-processing атмосферы и glow; - Visual Effect Graph требуется для particle systems звёздной короны (это относится к полной функциональности). ⁶

Для Lite корректная формулировка: URP + Unity 6 фактически требуются, потому что Lite включает атмосферный post-processing glow. ⁷ Требование Visual Effect Graph конкретно для Lite в открытой документации не подтверждено (VFX-корона указана как upgrade). Я не знаю, требуется ли Visual Effect Graph для Lite-пакета в вашей конкретной версии/сценах. ¹

Производительность, мерцание и управляемая деградация

Официальный performance-гайд фиксирует: - стоимость растёт с покрытием экрана объектом и с разрешением; - звёзды и газовые гиганты — наиболее тяжёлые из-за дорогих noise-функций/многослойного domain warp и процедурной фильтрации; - earth-like/луны/кольца обычно легче. ⁸

Есть официальные рекомендации по борьбе с мерцанием/алиасингом: baked textures (в платной версии), shader filtering и camera antialiasing (TAA). ⁹

Параметр `Filtering Detail` прямо описан как “увеличение сглаживания вдали ценой производительности”, и основан на технике фильтрации от Inigo Quilez ¹⁰ . ¹¹

Лицензия и юридическая “рамка использования”

Карточка ассета указывает Standard Unity Asset Store EULA и тип лицензии Extension Asset. ⁵ Для Extension Asset Unity разъясняет правило “seat per user с доступом к исходникам ассета”.

Доступные точки API/автоматизации из документации

Полезные “якоря” для автоматизации, которые прямо описаны: - Добавление планет через custom menu commands в Hierarchy; автоматически создаются материалы/ресурсы и GameObject со скриптами. ⁶

- Компонент `Procedural Body` (именно как концепт) имеет custom inspector кнопки генерации/рандомизации; параметры, заданные через компонент, override значения в материале. ⁶

- Post-processing glow реализован через Render Graph post-processing; названия скриптов: PostFXGlowEffect, PlanetGlow, StarCorona; планеты, созданные через menu items, получают настройку автоматически. ¹²

Чего не хватает для строгой автономности (и это нужно компенсировать инженерно): - нет публичного списка имён shader properties и точных диапазонов значений для Lite; - нет в документации описания “конфиг-файлов” ассета, которые можно было бы гарантированно генерировать.

Отсюда следует design-решение: **встроить в ваш конвейер авто-экспорт “схемы свойств материалов”** (имя/тип/Range-границы/дефолт), чтобы Claude мог обновлять планеты без ручного анализа Shader Graph.

Параметры, влияющие на внешний вид, и маппинг на аналоги планет Солнечной системы

Важная оговорка про “параметрические диапазоны”

Документация Lite перечисляет категории параметров (континенты, лёд, градиенты и так далее), но не публикует: - точные имена shader properties, - точные числовые диапазоны (min/max) для всех параметров Lite.

Поэтому ниже “диапазоны” заданы в виде: - либо нормализованных 0...1 там, где это естественно (доли, плотности), - либо множителей от template-дефолтов (×k), где числовой домен неизвестен.

Это сделано намеренно: в конвейере предлагается автоматический резолв диапазонов через Editor-интроспекцию материалов/шейдеров, чтобы переживать обновления ассета без ручного подбора.

Какие группы параметров управляют чем

Moons (Lite): - Appearance: procedural pattern + scale + gradient + saturation. ¹

Earth-like (Lite): - Continents: continent scale и land-ocean coverage ratio. ¹

- Ice: максимальный охват и “edge falloff”. ¹

- Surface color: noise scale, land gradient, saturation, ocean deep/shallow colors. ¹

- Atmosphere: Fresnel edge tint и atmospheric glow (color/size/falloff), плюс упоминается post-processing glow эффект. ¹³

Gas giant (Lite): - Bands: color gradient, band scale, flow speed; atmosphere via Fresnel. ¹

- Для дальних планов/слабых GPU: “Gas Giant (Simplified)”. ¹

Rings (Lite): - Shape: inner/outer radius, edge fade, banding pattern. ¹

- Lighting overrides: two-sided lighting, min brightness, light falloff sharpness. ¹

Stars (Lite): - Molten pattern: domain warp scale/intensity, pattern scale, flow speed. ¹

- Color sampling: primary gradient + Q/R modulation. ¹

- Corona effects: Fresnel + unlit spherical glow density/falloff. ¹

Биомы, температура, seed и LOD в контексте Lite

Биомы и температура: - Lite не заявляет отдельной климатической/биомной системы. Фактически “биомность” — это комбинация градиентов и шумов + доли суши/океана + льда. ¹

- Поэтому “температура” в Solar-маппинге реализуется как **производный художественный параметр**, который вы маппите на `IceCapExtent`, палитры (теплее/холоднее), интенсивность атмосферного края.

Seed (детерминизм): - В Lite заявлены инструменты рандомизации, но в документации Lite нет описания “seed-параметра” для этих инструментов. ¹

- Следовательно, для гарантированной воспроизводимости детерминизм лучше обеспечить “снаружи”: `seed` → канонические параметры → реальные shader properties.

LOD: - Поскольку это не terrain tool и не заявляет геометрический LOD планет, практический LOD строится через: (а) упрощённый gas giant; (б) управление `Filtering Detail` для далёких паттернов; (в) TAA; (г) включение/отключение post-processing glow на дистанциях. ¹⁴

Рекомендуемые целевые диапазоны канонических параметров

Здесь значения — **рекомендации для вашего автогенератора**, а не “официальные лимиты шейдера”.

Earth-like (Lite): - `LandOceanRatio` (0...1): океанический 0.05–0.30; землеподобный 0.25–0.40; сухой 0.85–0.98. - `ContinentScaleMul` (× дефолта): 0.6–2.5. - `IceCapExtent` (0...1): 0.0–0.6. - `IceEdgeFalloffMul` (× дефолта): 0.5–2.0. - `SurfaceNoiseScaleMul` (× дефолта): 0.5–3.0. - `Saturation` (0...1.2): 0.3–1.1. - `AtmosphereGlowSizeMul` (× дефолта): 0.7–2.5. - `AtmosphereGlowFalloffMul` (× дефолта): 0.6–2.0. ¹

Moon-like (Lite): - `SurfacePatternScaleMul`: 0.8–5.0. - `Saturation`: 0.0–0.4 (реализм) или до 1.0 (фантастика). ¹

Gas giant (Lite): - `BandScaleMul`: 0.6–3.0. - `FlowSpeedMul`: 0.3–2.5. - `FresnelEdgeIntensityMul`: 0.5–2.0. ¹

Rings (Lite): - `InnerRadius`, `OuterRadius`: как доли радиуса планеты, например 1.2R–2.5R. - `EdgeFadeStrength`: 0.2–2.0. - `RotationSpeedMul`: 0.3–2.0. - `MinBrightnessLevel`: 0.05–0.30. ¹

Stars (Lite): - `DomainWarpScaleMul`: 0.5–2.5. - `DomainWarpIntensityMul`: 0.5–3.0. - `MoltenPatternScaleMul`: 0.5–3.0. - `FlowSpeedMul`: 0.3–2.0. - `CoronaGlowDensityMul`: 0.6–2.5. ¹

Таблица: аналоги планет Солнечной системы → параметры Lite и ожидаемый вид

Легенда: - Все `...Mul` — множители относительно template-дефолта (1.0 = дефолт). - Цвета описаны как художественные палитры, так как Lite опирается на градиенты. ¹

Планета	Lite архетип	Рекомендуемый seed	Палитра (ориентир)	Ключевые параметры (диапазоны)	Кольца
Mercury	Moon-like	<code>hash("mercury")</code>	тёмно-серый → светло-серый	<code>SurfacePatternScaleMul</code> 2.0–4.0 , <code>Saturation</code> 0.05–0.25	нет
Venus	Earth-like (имитация)	<code>hash("venus")</code>	кремово-жёлтый/белый (как облака)	<code>LandOceanRatio</code> 0.90–1.00 , <code>ContinentScaleMul</code> 1.5–2.5 , <code>SurfaceNoiseScaleMul</code> 0.6–1.2 , <code>AtmosphereGlowSizeMul</code> 1.8–2.5	нет
Earth	Earth-like	<code>hash("earth")</code>	океан: синий + бирюза; суша: зелёный→коричневый; лёд белый	<code>LandOceanRatio</code> 0.25–0.40 , <code>ContinentScaleMul</code> 0.9–1.3 , <code>IceCapExtent</code> 0.15–0.35 , <code>AtmosphereGlowSizeMul</code> 1.0–1.6	нет
Mars	Earth-like	<code>hash("mars")</code>	охра/красный; океан почти отсутствует	<code>LandOceanRatio</code> 0.92–0.99 , <code>IceCapExtent</code> 0.05–0.15 , <code>AtmosphereGlowSizeMul</code> 0.6–1.0	нет
Jupiter	Gas giant	<code>hash("jupiter")</code>	бежево-оранжевые/коричневые полосы	<code>BandScaleMul</code> 1.6–2.6 , <code>FlowSpeedMul</code> 0.9–1.8 , <code>FresnelEdgeIntensityMul</code> 0.7–1.2	нет
Saturn	Gas giant + rings	<code>hash("saturn")</code>	газ: бледно-жёлтый; кольца: светло-серые/беж	Газ: <code>BandScaleMul</code> 1.0–1.8 , <code>FlowSpeedMul</code> 0.6–1.2 ; Кольца: <code>InnerRadius</code> ~1.2R , <code>OuterRadius</code> ~2.3R , <code>MinBrightnessLevel</code> 0.10–0.25	да

Планета	Lite архетип	Рекомендуемый seed	Палитра (ориентир)	Ключевые параметры (диапазоны)	Коль
Uranus	Gas giant	<code>hash("uranus")</code>	светло-циановый, низкий контраст	<code>BandScaleMul 0.6-1.2</code> , <code>FlowSpeedMul 0.3-0.8</code>	нет
Neptune	Gas giant	<code>hash("neptune")</code>	насыщенный синий, чуть более контрастный	<code>BandScaleMul 1.0-1.8</code> , <code>FlowSpeedMul 0.7-1.6</code>	нет
Pluto	Moon-like	<code>hash("pluto")</code>	серо-коричневый + белые ледяные пятна	<code>SurfacePatternScaleMul 1.2-2.6</code> , <code>Saturation 0.1-0.5</code>	нет

Automated pipeline для Claude: входы, выходы, скрипты, CI hooks, bundles

Архитектура “минимум ручных правок при обновлениях”

Критическая идея: **не зависеть от Editor-кнопок ассета**, а использовать ассет как “рендер-движок” (шейдеры/материалы/префабы-шаблоны), а все решения хранить в ваших данных (Planet Spec). Тогда при обновлениях кода вы просто пересобираете. При изменениях ассета вы регенерируете “schema” и либо автоматически адаптируетесь, либо включаете fallback.

Рекомендуемая структура репозитория: - `Assets/PlanetGen/Templates/` — TemplatePrefabs (MoonLiteTemplate, EarthLikeLiteTemplate, GasGiantLiteTemplate, GasGiantLiteSimplifiedTemplate, RingLiteTemplate, StarLiteTemplate). Эти префабы создаются один раз (вручную) и далее только используются. - `Assets/PlanetGen/Specs/*.json` — Planet Spec файлы (единственный “источник истины” для визуальных параметров). - `Assets/PlanetGen/Generated/` — автогенерируемые префабы/материалы/градиенты (перегенерируются полностью). - `Assets/PlanetGen/Schema/` — автогенерируемые JSON (схема shader properties по каждому архетипу + версии ассета/Unity).

Входы/выходы утилиты генерации

Входы: - Планеты (список): `id`, `archetype`, `seed`, `solarAnalog` (опционально), канонические параметры. - TemplatePrefabs. - Правила трансформации (SolarAnalog → canonical params), если Claude генерирует “по аналогам”.

Выходы: - Один префаб на планету (например `Assets/PlanetGen/Generated/Planets/earth.prefab`). - Материалы (как отдельные `.mat`), возможно ScriptableObjects/Gradients (если

используются). - `planet_manifest.json` (id → адрес/путь, тип, радиус, набор “тэгов” для геймплея). - Скриншоты-снимки для визуальной регрессии (в CI — как артефакты).

How Claude “бежит” по пайплайну

С точки зрения автоматизации Claude делает четыре операции: 1) Обновить/создать `*.json` specs (либо вручную, либо автоматически на основе SolarAnalog mapping). 2) Запустить Unity batchmode метод генерации. 3) Запустить тесты (Unity Test Framework) и визуальную валидацию. 4) Собрать bundles/addressables.

Для bundles: - Если проект уже на Addressables, генератор должен пометить generated prefabs как Addressable (группы по архетипу/биому/системе). - Если Addressables не используются, можно собирать AssetBundles стандартным билдом Unity, но Addressables обычно удобнее для patch-обновлений.

Mermaid flowchart конвейера

```
graph TD
    A[Planet Specs JSON/YAML] --> B[Claude updates/creates specs]
    B --> C[Commit or PR]
    C --> D[CI job: Unity batchmode]
    D --> E[Schema export: shader properties + versions]
    E --> F[Generate/Update prefabs + materials deterministically]
    F --> G[Render snapshots for visual regression]
    G --> H[Run unit/integration tests]
    H --> I{All checks pass?}
    I -- No --> J[Fallback: keep previous Generated + report diffs/errors]
    I -- Yes --> K[Build Addressables/AssetBundles]
    K --> L[Publish artifacts + update planet_manifest.json]
```

Editor/runtime скрипты, псевдокод, обработка обновлений и аварийные стратегии

Какие скрипты нужны

Editor-скрипты: - `PlanetSchemaExporter` — экспортирует “схему” всех свойств материалов шаблонов (имя свойства, тип, Range-границы, дефолт). - `PlanetBatchGenerator` — читает Specs, клонирует TemplatePrefab, применяет параметры, сохраняет материалы/префабы, создаёт/обновляет manifest. - `SnapshotRenderer` — рендерит планеты в фиксированной сцене/камере в PNG для регрессии.

Runtime-скрипты: - `PlanetCatalog` — читает `planet_manifest.json`. - `PlanetLoader` — грузит префабы (Addressables/Resources/Bundles) и инстанцирует. - `PlanetFallbackPolicy` — если планета не загрузилась или шейдеры не поддерживаются, выбирает fallback (например, простая сфера с цветом).

Псевдокод: batch генерация

```
// English-only comments as requested.

public static class PlanetGenCli
{
    // Entry point for Unity -batchmode -executeMethod
    public static int GenerateAll()
    {
        try
        {
            var schema = PlanetSchemaExporter.ExportFromTemplates("Assets/
PlanetGen/Templates", "Assets/PlanetGen/Schema");
            var specs = PlanetSpecLoader.LoadAll("Assets/PlanetGen/Specs");

            foreach (var spec in specs)
            {
                PlanetGenerator.GenerateOne(spec, schema,
                    templatesRoot: "Assets/PlanetGen/Templates",
                    outputRoot: "Assets/PlanetGen/Generated");
            }

            PlanetManifestWriter.Write("Assets/PlanetGen/Generated/
planet_manifest.json", specs);
            AssetDatabase.SaveAssets();
            AssetDatabase.Refresh();
            return 0;
        }
        catch (Exception ex)
        {
            // Log for CI parsing.
            UnityEngine.Debug.LogError($"PlanetGen failed: {ex}");
            return 1;
        }
    }
}

public static class PlanetGenerator
{
    public static void GenerateOne(PlanetSpec spec, Schema schema, string
templatesRoot, string outputRoot)
    {
        // Clone template prefab.
        var templatePath = TemplateResolver.Resolve(spec.Archetype,
templatesRoot);
        var go = PrefabUtility.LoadPrefabContents(templatePath);

        // Ensure we work on instance materials, not sharedMaterial.
        var renderer = go.GetComponentInChildren<Renderer>(true);
        if (renderer == null) throw new InvalidOperationException("Renderer
not found in template.");
    }
}
```

```

var mat = UnityEngine.Object.Instantiate(renderer.sharedMaterial);
renderer.sharedMaterial = mat;

// Deterministic parameter generation.
var canonical =
CanonicalParamDeriver.Derive(spec); // Uses spec.Seed + optional SolarAnalog
mapping.

// Apply using schema mapping + robust resolver.
ShaderPropertyApplier.Apply(mat, canonical, schema);

// Save material + prefab atomically.
var planetDir = $"{outputRoot}/Planets/{spec.Id}";
DirectoryUtil.EnsureEmptyDir(planetDir + "/_tmp");

AssetDatabase.CreateAsset(mat, $"{planetDir}/Planet.mat");
PrefabUtility.SaveAsPrefabAsset(go, $"{planetDir}/Planet.prefab");
PrefabUtility.UnloadPrefabContents(go);

DirectoryUtil.AtomicSwap(planetDir + "/_tmp", planetDir);
}
}

```

Ключевые моменты, которые защищают от “ломающихся обновлений”: - Не трогать sharedMaterial напрямую; всегда инстанцировать материал (иначе изменения протекут в шаблоны и/или поведение будет зависеть от Editor/Runtime различий — похожая проблема упоминалась в контексте теней в демо/обсуждениях).

- Генерировать в temp-директорию и делать atomic swap, чтобы при падении не оставить полусгенерированный мусор. - Хранить templates отдельно и не модифицировать их.

Псевдокод: экспорт схемы свойств и резолвер имён

```

// English-only comments as requested.

public static class PlanetSchemaExporter
{
    public static Schema ExportFromTemplates(string templatesRoot, string
schemaOutDir)
    {
        var schema = new Schema();

        foreach (var templatePath in AssetDatabase.FindAssets("t:Prefab",
new[] { templatesRoot })
                .Select(AssetDatabase.GUIDToAssetPath))
        {
            var prefab =
AssetDatabase.LoadAssetAtPath<GameObject>(templatePath);
            var renderer = prefab.GetComponentInChildren<Renderer>(true);
            if (renderer == null) continue;

```

```

        var mat = renderer.sharedMaterial;
        if (mat == null || mat.shader == null) continue;

        var key = prefab.name;
        schema.Archetypes[key] = ShaderIntrospection.Dump(mat.shader,
mat);
    }

    File.WriteAllText($"{schemaOutDir}/schema.json",
JsonUtility.ToJson(schema, true));
    return schema;
}
}

public static class ShaderPropertyApplier
{
    public static void Apply(Material mat, CanonicalParams p, Schema schema)
    {
        // Robust approach:
        // 1) Try exact mapping from schema (maintained by your project).
        // 2) If not found, try fuzzy match by token sets (e.g., "continent",
"ratio", "ice").
        // 3) If still not found, keep material default and log warning.

        ApplyFloat(mat, schema, "LandOceanRatio", p.LandOceanRatio);
        ApplyColorGradient(mat, schema, "LandGradient", p.LandGradient);
        ApplyColor(mat, schema, "OceanDeepColor", p.OceanDeepColor);
        // ...
    }
}
}

```

Это закрывает главную дыру Lite-документации: отсутствие списка имён shader properties. При обновлении ассета меняется шейдер/поля — схема переснимается, и резолвер либо адаптируется, либо сообщает конкретные diffs.

Стратегии восстановления после ошибок и изменений API

Наиболее вероятные классы поломок и автоматические реакции:

1) “Свойство не найдено / переименовано”

- Поведение: лог “Missing property”, пропуск применения, оставить дефолт.
- Валидация: визуальная регрессия увидит отклонение, CI провалит пайплайн, но при этом старые Generated ассеты останутся (atomic swap).

2) “Мерцание/алиасинг при дальнем виде”

- Авто-правка: поднять `Filtering Detail` на звёздах/газовых гигантах в пределах бюджета GPU. ¹⁵
- Альтернатива: включить TAA в тестовой и продуктовой камере. ¹⁶

3) “Просадка производительности при больших размерах на экране / 4K”

- Авто-правка: снижать интенсивность фильтрации, переключать дальние планеты на “Gas Giant (Simplified)”, отключать post-processing glow на дистанции. ¹⁷

4) “Известные проблемы в Editor” (например, падение производительности при выделенном объекте в инспекторе)

- Это влияет на интерактивную работу, но batchmode/билды должны быть стабильнее; в пайплайне важно тестировать в build-режиме, не в Editor-view. ¹⁸

Также полезно опираться на changelog: он показывает, что состав компонентов может меняться (например, удаление конкретных компонентов) — это сигнал держать вашу интеграцию через “данные + интроспекция”, а не через жёсткие ссылки на типы. ¹⁹

План тестирования, валидации и интеграции в космосим с fallback при изменениях ассета

Визуальная валидация

Цель: обнаружить “тихие” изменения шейдеров/диапазонов/градиентов после обновлений кода или ассета.

Рекомендуемый набор: - Snapshot-рендер каждого planet prefab в фиксированном окружении (камера, экспозиция, bloom включён/выключен одинаково). - Сравнение с эталоном: - простой pixel diff с порогом, - или SSIM/перцептивная метрика (если хотите меньше ложных срабатываний). - Отдельные “канареечные” планеты: одна Earth-like, один gas giant, одна star, одно ring, потому что они покрывают разные ветки параметров. ¹

Производительная валидация

Из официального гайда следует, что самые тяжёлые случаи — когда звезда/газовый гигант занимает большой процент экрана; также 4K резко усиливает нагрузку. ⁸ Поэтому тесты полезно строить вокруг worst-case сцен:

- Сцена “WorstCaseStarFullScreen”: звезда на весь экран, измерение frame time в фиксированном билде.
- Сцена “WorstCaseGasGiantFullScreen”: gas giant на весь экран.
- Сцена “TypicalSolarMap”: типичная для вашей игры дистанция (малый экранный размер планет), чтобы проверить, что фильтрация не завышена.

Lite не даёт “baked textures tools” как функцию (это описано как upgrade), поэтому регрессия производительности должна отслеживаться особенно внимательно (процедурные шумы дороже текстур). ²⁰

Unit и интеграционные тесты

Unit tests (быстрые): - Парсинг PlanetSpec. - Детерминизм: одинаковый seed → одинаковые canonical params. - Резолвер свойств: если schema изменилась, тест должен даунгрейдиться в “warning mode” и выдавать отчёт diff, а не падать без объяснения.

Integration tests: - Загрузка Addressable/AssetBundle префабов (если используется). - Проверка fallback: если префаб не найден, подставить "generic sphere материал" и продолжить игру. - Проверка корректности "instance materials vs sharedMaterial" (чтобы избежать "статических" артефактов при рандомизации/обновлениях).

Integration checklist для космосима

Совместимость/рендер: - URP проект и Unity 6 (ориентир по карточке ассета). ²¹
- Настроить TAA в камере, если наблюдается мерцание процедурных деталей. ¹⁶
- Для далёких объектов — аккуратно применять **Filtering Detail** (качество/стоимость) и избегать чрезмерных значений при крупных планетах на экране. ¹⁵
- Post-processing glow: понимать, что это Render Graph post-processing (custom pass + fullscreen shader) и что он требует корректной конфигурации материалов/скриптов. ¹²

Контент-пайплайн: - TemplatePrefabs не редактируются руками после первичной настройки. - Generated папка полностью пересобирается из Specs. - Schema экспортируется на CI при каждом прогоне и хранится как артефакт/файл в репо.

Рекомендуемые fallback-поведения при изменениях API ассета

Цель: игра не должна "падать" из-за косметики.

Fallback уровни: 1) Если не найдены некоторые shader properties — оставить дефолт шаблона, логировать diff. 2) Если шейдер не найден/не компилируется — заменить материал на "Unlit/Color" (или ваш простой planet shader) и выставить базовый цвет по PlanetSpec. 3) Если не работает post-processing glow — отключить glow-компоненты и оставить URP bloom (документация прямо описывает glow как отдельный от bloom эффект и допускает использование bloom). ¹²
4) Если проблемы производительности — переключать газовые гиганты на "Simplified", снижать фильтрацию и отключать дорогие эффекты на дистанции. ²⁰

Mermaid timeline интеграционных задач

```
gantt
    title Integration timeline for autonomous planet generation
    dateFormat YYYY-MM-DD
    axisFormat %d.%m

    section Project setup
    URP + Unity 6 baseline           :a1, 2026-03-12, 5d
    Import Lite + verify sample scenes: a2, after a1, 3d

    section Generator foundations
    Create TemplatePrefabs          :b1, after a2, 3d
    Implement SchemaExporter         :b2, after b1, 4d
    Implement BatchGenerator         :b3, after b2, 6d

    section Validation
    Snapshot rendering + diff        :c1, after b3, 5d
    Performance worst-case scenes    :c2, after c1, 4d
```

Unit + integration tests :c3, after c1, 6d

section Build & runtime integration

Addressables/AssetBundles build :d1, after c3, 4d

Runtime loader + fallback policy :d2, after d1, 4d

CI hooks + nightly regen :d3, after d2, 3d

1 3 7 10 13 14 20 **Lite Samples Pack - Procedural Planet Generation - Parallel Cascades**

https://parallelcascades.com/procedural-planet-generation/latest/lite-samples-pack/?utm_source=chatgpt.com

2 4 **Procedural Planet Generation**

https://parallelcascades.com/procedural-planet-generation/latest/?utm_source=chatgpt.com

5 21 **Procedural Planet Generation - Lite: Baked Planets and Simple Procedural Shaders | VFX Shaders | Unity Asset Store**

https://assetstore.unity.com/packages/vfx/shaders/procedural-planet-generation-lite-baked-planets-and-simple-proce-296362?srsId=AfmBOooqabuMHMAD1qFoIRy-2qhKwozQit0VbFacBSbmXzgK5kRR3iD1&utm_source=chatgpt.com

6 **Get Started - Procedural Planet Generation**

https://parallelcascades.com/procedural-planet-generation/latest/get-started/?utm_source=chatgpt.com

8 17 18 **System Requirements - Procedural Planet Generation**

https://parallelcascades.com/procedural-planet-generation/latest/system-requirements-and-performance-optimisation/?utm_source=chatgpt.com

9 16 **Camera Antialiasing - Procedural Planet Generation**

https://parallelcascades.com/procedural-planet-generation/latest/antialiasing/?utm_source=chatgpt.com

11 15 **Shader Filtering - Procedural Planet Generation**

https://parallelcascades.com/procedural-planet-generation/latest/filtering/?utm_source=chatgpt.com

12 **Post Processing Glow - Procedural Planet Generation**

https://parallelcascades.com/procedural-planet-generation/latest/post-processing-glow/?utm_source=chatgpt.com

19 **Changelog - Procedural Planet Generation**

https://parallelcascades.com/procedural-planet-generation/latest/changelog/?utm_source=chatgpt.com